

rabbit and redis

Практика: RabbitMQ и Redis как брокеры сообщений

Цель: сравнить RabbitMQ и Redis в одинаковых условиях: один формат сообщения, одно число producers/consumers, одна длительность теста и одинаковые лимиты CPU/RAM.

Стенд

- producer и consumer: bench.py
- RabbitMQ: обычная очередь
- Redis: список RPush / BLPOP
- лимиты брокеров: 1 CPU, 512 MB RAM
- формат сообщения: id|sent_at_ns|payload

Redis проверяется как простой брокер-очередь. Для задачи этого достаточно: producer кладет сообщения, consumer читает их, стенд считает задержку и остаток очереди.

Запуск

```
docker compose up -d rabbitmq redis

docker compose build runner

docker compose run --rm runner bash scripts/run_matrix.sh
```

RabbitMQ UI: <http://localhost:15672>

Логин и пароль: guest / guest

Что измеряется

Результаты пишутся в results/results.csv

Поля:

- sent — отправлено producerами
- processed — прочитано consumerами
- backlog — сколько осталось в очереди после прогона
- lost — отправлено, но не обработано и не осталось в очереди
- errors — ошибки отправки или обработки
- avg_ms — средняя задержка
- p95_ms — 95-я перцентиль задержки
- max_ms — максимальная задержка

Прогоны

Базовое сравнение:

```
docker compose run --rm runner bash -lc \  
  
'BROKERS="rabbit redis" SIZES="1024" RATES="1000" DURATION=20 bash scripts/run_matrix.sh'
```

Влияние размера сообщения:

```
docker compose run --rm runner bash -lc \  
  
'BROKERS="rabbit redis" SIZES="128 1024 10240 102400" RATES="1000" DURATION=20 bash
```

```
scripts/run_matrix.sh'
```

Влияние интенсивности:

```
docker compose run --rm runner bash -lc \  
'BROKERS="rabbit redis" SIZES="1024" RATES="1000 5000 10000" DURATION=20 bash scripts/run_matrix.sh'
```

Полная матрица по умолчанию:

```
docker compose run --rm runner bash scripts/run_matrix.sh
```

Дополнительные замеры

Во время прогона можно открыть отдельный терминал.

CPU и RAM:

```
docker stats broker-rabbitmq broker-redis
```

Очередь RabbitMQ:

```
docker compose exec rabbitmq rabbitmqctl list_queues name messages
```

Очередь Redis:

```
docker compose exec redis redis-cli LLEN broker_bench
```

Таблица результатов

Прогон: 1 producer, 1 consumer, длительность каждого теста 20s

broker	size	target msg/sec	actual msg/sec	sent	processed	backlog	lost	errors	avg ms	p95 ms	max ms
RabbitMQ	128 B	1000	989	19984	19984	0	0	0	0.8	0.97	3.63
RabbitMQ	128 B	5000	2447	49450	49450	0	0	0	1.05	1.58	4.32
RabbitMQ	128 B	10000	2491	50315	50315	0	0	0	1.03	1.55	10.63
RabbitMQ	1 KB	1000	989	19984	19984	0	0	0	0.8	1.03	3.56
RabbitMQ	1 KB	5000	2502	50544	50544	0	0	0	1.02	1.56	4.56
RabbitMQ	1 KB	10000	2423	48936	48936	0	0	0	1.09	1.67	5.57
RabbitMQ	10 KB	1000	989	19984	19984	0	0	0	0.78	0.99	4.01
RabbitMQ	10 KB	5000	2213	44699	44699	0	0	0	1.08	1.64	6.5
RabbitMQ	10 KB	10000	2238	45212	45212	0	0	0	1.15	1.74	7.75

broker	size	target msg/sec	actual msg/sec	sent	processed	backlog	lost	errors	avg ms	p95 ms	max ms
RabbitMQ	100 KB	1000	989	19984	19984	0	0	0	2.8	5.02	23.99
RabbitMQ	100 KB	5000	891	20004	17908	1904	192	0	2310.02	3174.04	3432.14
RabbitMQ	100 KB	10000	935	20853	18841	1803	209	0	2730.49	3853.31	4090.5
Redis	128 B	1000	950	20000	20000	0	0	0	0.35	0.42	9.47
Redis	128 B	5000	4998	99977	99970	7	0	0	74.21	193.03	233.5
Redis	128 B	10000	6779	135582	135582	0	0	0	19.95	59.59	82.62
Redis	1 KB	1000	951	20000	20000	0	0	0	0.35	0.43	6.87
Redis	1 KB	5000	4999	99990	99988	2	0	0	80.67	188.26	207.93
Redis	1 KB	10000	6822	137045	136439	606	0	0	76.9	110.04	150.48
Redis	10 KB	1000	950	20000	20000	0	0	0	0.41	0.49	7.21
Redis	10 KB	5000	1843	38860	38860	0	0	0	0.39	0.46	10.49
Redis	10 KB	10000	1804	38033	38033	0	0	0	0.4	0.47	10.54
Redis	100 KB	1000	951	20000	20000	0	0	0	0.68	0.83	11.3
Redis	100 KB	5000	1269	26678	26678	0	0	0	0.85	1.07	11.11

broker	size	target msg/sec	actual msg/sec	sent	processed	backlog	lost	errors	avg ms	p95 ms	max ms
Redis	100 KB	10000	1288	27094	27094	0	0	0	0.82	1.03	11.37

```
{'broker': 'rabbit', 'payload_size': 128, 'target_rate': 1000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 19984, 'processed': 19984, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 0.83, 'p95_ms': 1.08, 'max_ms': 25.47}
{'broker': 'rabbit', 'payload_size': 128, 'target_rate': 5000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 54381, 'processed': 54381, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 1.06, 'p95_ms': 1.84, 'max_ms': 10.21}
{'broker': 'rabbit', 'payload_size': 128, 'target_rate': 10000, 'duration_sec': 20.21, 'producers': 1, 'consumers': 1, 'sent': 50554, 'processed': 50554, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 1.05, 'p95_ms': 1.62, 'max_ms': 7.67}
{'broker': 'rabbit', 'payload_size': 1024, 'target_rate': 1000, 'duration_sec': 20.21, 'producers': 1, 'consumers': 1, 'sent': 19984, 'processed': 19984, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 0.82, 'p95_ms': 1.23, 'max_ms': 3.82}
{'broker': 'rabbit', 'payload_size': 1024, 'target_rate': 5000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 48072, 'processed': 48072, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 1.14, 'p95_ms': 1.88, 'max_ms': 7.71}
{'broker': 'rabbit', 'payload_size': 1024, 'target_rate': 10000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 49463, 'processed': 49463, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 1.08, 'p95_ms': 1.72, 'max_ms': 6.19}
{'broker': 'rabbit', 'payload_size': 10240, 'target_rate': 1000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 19985, 'processed': 19985, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 0.8, 'p95_ms': 1.18, 'max_ms': 3.98}
{'broker': 'rabbit', 'payload_size': 10240, 'target_rate': 5000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 46011, 'processed': 46011, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 1.09, 'p95_ms': 1.71, 'max_ms': 7.35}
{'broker': 'rabbit', 'payload_size': 10240, 'target_rate': 10000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 68499, 'processed': 68499, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 0.88, 'p95_ms': 1.85, 'max_ms': 11.46}
{'broker': 'rabbit', 'payload_size': 102400, 'target_rate': 1000, 'duration_sec': 20.2, 'producers': 1, 'consumers': 1, 'sent': 19984, 'processed': 19984, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 2.12, 'p95_ms': 3.46, 'max_ms': 22.57}
{'broker': 'rabbit', 'payload_size': 102400, 'target_rate': 5000, 'duration_sec': 20.66, 'producers': 1, 'consumers': 1, 'sent': 20633, 'processed': 18709, 'backlog': 1583, 'lost': 341, 'errors': 0, 'avg_ms': 2725.07, 'p95_ms': 5280.25, 'max_ms': 5412.37}
{'broker': 'rabbit', 'payload_size': 102400, 'target_rate': 10000, 'duration_sec': 21.18, 'producers': 1, 'consumers': 1, 'sent': 22629, 'processed': 21806, 'backlog': 629, 'lost': 194, 'errors': 0, 'avg_ms': 2691.65, 'p95_ms': 4066.36, 'max_ms': 4701.74}
{'broker': 'redis', 'payload_size': 128, 'target_rate': 1000, 'duration_sec': 21.1, 'producers': 1, 'consumers': 1, 'sent': 20000, 'processed': 20000, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 0.36, 'p95_ms': 0.52, 'max_ms': 7.71}
{'broker': 'redis', 'payload_size': 128, 'target_rate': 5000, 'duration_sec': 21.1, 'producers': 1, 'consumers': 1, 'sent': 99985, 'processed': 99985, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 33.59, 'p95_ms': 80.44, 'max_ms': 95.32}
{'broker': 'redis', 'payload_size': 128, 'target_rate': 10000, 'duration_sec': 21.05, 'producers': 1, 'consumers': 1, 'sent': 133348, 'processed': 133348, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 24.0, 'p95_ms': 73.34, 'max_ms': 109.75}
{'broker': 'redis', 'payload_size': 1024, 'target_rate': 1000, 'duration_sec': 21.05, 'producers': 1, 'consumers': 1, 'sent': 19999, 'processed': 19999, 'backlog': 0, 'lost': 0, 'errors': 0, 'avg_ms': 0.37, 'p95_ms': 0.45, 'max_ms': 10.41}
```

Выводы

- Большую пропускную способность показал Redis: максимум 6822 msg/sec на 1 KB, у RabbitMQ максимум 2502 msg/sec
- Маленькие сообщения лучше обработал Redis: на 128 B он дошел до 6779 msg/sec, RabbitMQ остановился около 2491 msg/sec
- На 100 KB оба брокера просели по throughput, но RabbitMQ начал копить очередь и терять сообщения из теста
- RabbitMQ начал деградировать на 100 KB при цели 5000 msg/sec: backlog = 1904, p95 = 3174.04 ms
- Redis начал деградировать по задержке на 128 B и 1 KB при цели 5000 msg/sec, а очередь появилась на 1 KB при цели 10000 msg/sec: backlog = 606

Деградацией считаю ситуацию, где растёт backlog, заметно поднимается p95_ms или появляются ошибки

Для этого сценария удобнее собственный скрипт, а не k6/Locust: он напрямую работает с RabbitMQ и Redis, держит один формат сообщения и дает одинаковые параметры запуска для двух брокеров.